Consistency Models and the CAP Theorem: Lab Report

Akylbek Askhat Serikuly
Digital Engineering, Narxoz University
CSS4004 Distributed Systems
Lab 6-7: Immutability, PACELC Trade-offs, and Quorum Tuning
9 November 2026

Consistency Models and the CAP Theorem: Lab Report

This lab extends the replicated gRPC service of Assignment 2 to a key-value store whose consistency is chosen per request. A write is either `QUORUM_CP` (a majority of the three replicas must acknowledge it) or `ALWAYS_AP` (any reachable replica accepts it). A read is either `LINEARIZABLE` (answered by the leaseholder) or `FOLLOWER_STALE` (answered by the replica that receives it, within a staleness bound). The network partitions are real: every replica is a separate process, and every connection between two replicas passes through a TCP proxy that a test can cut. This report records how each behaviour was tested, the quorum verification, the PACELC measurements and the answers to the conceptual questions. Every claim points to a test, a table of measured values or a captured log in the appendices.

System under test

Leaseholder. Node 1 (replica A, port 50051) holds the lease for the key range [a-z]; a key is in the range if its first character is a lowercase letter. Keys outside the range are rejected.

`QUORUM_CP` **write.** A follower that receives the write forwards it to A. A stamps it with its hybrid logical clock (wall time in microseconds that never runs backwards and moves past every timestamp it has seen) and sends `Prepare` to both followers in parallel; its own copy counts as the first acknowledgement. With two of three acknowledgements the write commits on A and reaches the followers with the next anti-entropy message, which A sends at once. With fewer the write is aborted: the followers drop their staged copy and the client receives `success = false` with `ReplicaUnavailableError: quorum lost`. An aborted write never becomes visible.

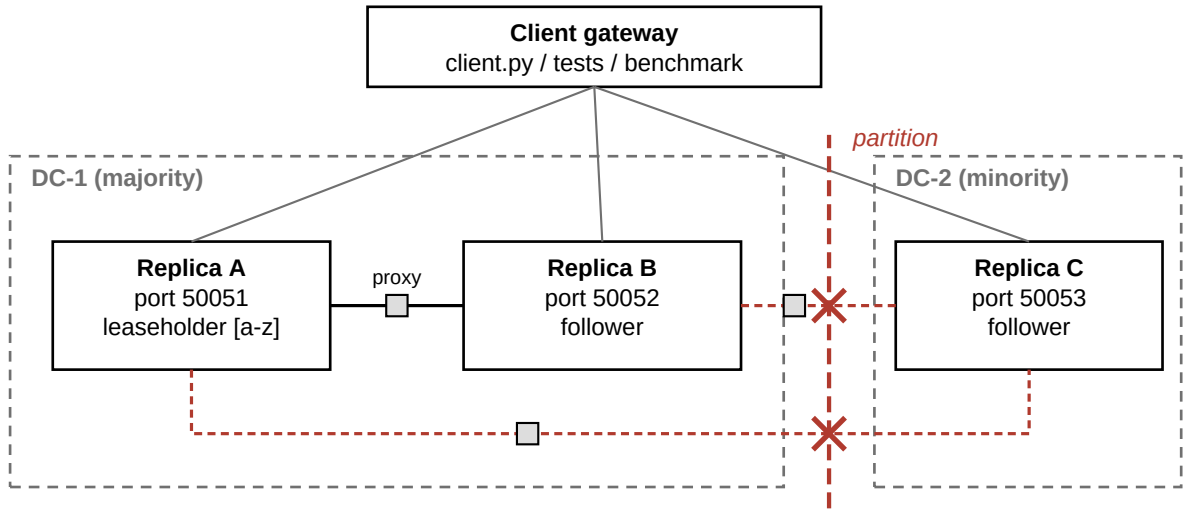
`ALWAYS_AP` **write.** Any replica applies the write at once with a timestamp and a vector clock (for every node, the time of its last write to this key), pushes it to the peers it can reach, and reports in `acks_received` how many replicas hold it.

Reads. A `LINEARIZABLE` read is answered only by A; every other replica proxies it to A, and if A cannot be reached the read fails with `LEASEHOLDER_UNAVAILABLE`. A `FOLLOWER_STALE` read is answered by the follower from its own store. Every 100 ms A sends each follower a closed timestamp: all committed writes before that time are already on the follower. If the follower lags more than `max_staleness_ms` behind it answers `STALE_READ_EXCEEDED`; otherwise it replies with `read_timestamp` = closed timestamp and `is_stale = true`.

Anti-entropy and conflicts. Every 100 ms each replica pushes to each peer the versions that changed since that peer's last acknowledgement. This one mechanism delivers commits, catches up a restarted replica and merges the two sides after a partition. Two versions are concurrent if neither vector clock contains the other. Last-write-wins (the default) keeps the version with the higher timestamp, the node id breaking ties; union merge keeps the items of both.

Figure 1

Test Setup: Replica Processes Connected Through Link Proxies, With the DC-1 | DC-2 Partition



Note. Every connection between two replicas passes through a TCP proxy in the test process, one per direction (grey squares). The partition closes the proxies that cross the red line and refuses new connections; the client keeps its direct connections to every replica.

Table 1

Lab Tasks and Where They Are Implemented

Task	Requirement	Implementation	Verified by
1	`kvstore.proto` with ReadConsistency, WriteStrategy, Get and Put	The public part of `kvstore.proto` is the handout's definition unchanged. A second service, `Replica`, carries the internal protocol: `Prepare`, `Sync` (anti-entropy and closed timestamp), `ReadLocal` and `Status`.	all tests
2.1	Node 1 (port 50051) is the leaseholder for [a-z]	`server.py`: `KEY_RANGES = [("a", "z")]`, leaseholder A; other keys give `InvalidKeyError`.	C4
2.2	`LINEARIZABLE` reads go strictly to the leaseholder	Followers proxy the Get to A over the replica network; A answers from its committed state (`is_stale = false`). A unreachable: gRPC `UNAVAILABLE`, `LEASEHOLDER_UNAVAILABLE`.	R1, R2
2.3	`FOLLOWER_STALE` served locally; `STALE_READ_EXCEEDED` when the follower lags	Lag = now - closed timestamp. Lag <= `max_staleness_ms`: local answer, `is_stale = true`. Otherwise gRPC `FAILED_PRECONDITION`, `STALE_READ_EXCEEDED`. `max_staleness_ms = 0` means 5000 ms.	R3, R4, R5
3	Partition DC-1 {A, B} | DC-2 {C}; CP and AP writes; LWW after recovery	`NetworkPartition` wrapper in `tests/test_cap.py` over the link proxies of `netsim.py`; the replicas are subprocesses started by `cluster.py`.	P1-P8
4	1,000 reads per scenario; median, p95, availability	`perf_cap_bench.py`	Tables 6-8
Obj. 1	R + W > N gives strong reads, R + W <= N stale reads	Write confined to W replicas by a partition, quorum read from every set of R replicas.	Q1-Q9, Table 5

1. Test Results and Quorum Verification

Table 2
Test Environment

Item	Value
Operating system	Microsoft Windows 11 Home, version 10.0.26200 (build 26200), 64-bit
Machine	ASUS ROG Strix G614JI laptop, Intel 64 Family 6 Model 183 processor (about 2.6 GHz), 32 GB RAM
Python	CPython 3.14.6, virtual environment (.venv in the repository root)
Packages	grpcio 1.84.0, grpcio-tools 1.84.0, protobuf 7.36.2, pytest 9.1.1
How the tests run	The test module starts three replica processes (python server.py) on free ports and six link proxies inside the pytest process. Tests that need another configuration (union merge, clock skew) start a cluster of their own. Nothing has to be started by hand.
Commands	pytest -v, python perf_cap_bench.py, python demo.py (all in the lab6_7 folder)

Failure injection

The wrapper `NetworkPartition` in `tests/test_cap.py` combines both techniques named in the handout. The replicas run as subprocesses, and the socket traffic between them is intercepted: replica A does not connect to B, it connects to a proxy that forwards the bytes to B. Entering the wrapper closes every open connection that crosses the line between the two data centres and refuses new ones, so the replicas see the same thing as in a real partition: the peer is unreachable. Leaving the wrapper restores the links, which is the partition recovery. A crash is simulated by killing the replica process; a restart starts a new process with an empty store.

Result of the run: **26 passed in 8.92s**. The full output is in Appendix B.

Table 3
Test Cases With Expected and Actual Results

Id	Test	Scenario	Expected result	Actual result	Result
P1	`test_cp_write_on_majority_side_succeeds`	Partition {A, B} | {C}. `QUORUM_CP` via A, then via B.	success = true, acks_received = 2 for both; A and B hold the value, C does not	success = true, acks_received = 2 for both; A and B hold "via-B", C: not found	pass
P2	`test_cp_write_on_minority_side_fails`	Same partition. `QUORUM_CP` via C.	success = false, acks_received < 2, `ReplicaUnavailableError` (quorum lost); no replica has the key	success = false, acks_received = 0, error `ReplicaUnavailableError` (quorum lost, leaseholder A is unreachable from C); key absent on A, B and C	pass
P3	`test_isolated_leaseholder_cannot_commit_alone`	Partition {B, C} | {A}. `QUORUM_CP` via A, then via B.	via A: failure, acks_received = 1; via B: failure (the lease stays on A)	via A: "quorum lost - 1/3 replicas acknowledged, 2 required"; via B: `ReplicaUnavailableError`	pass

Id	Test	Scenario	Expected result	Actual result	Result
P4	`test_ap_write_on_minority_side_is_accepted`	Partition {A, B} | {C}. `ALWAYS_AP` via C.	success = true, acks_received = 1; only C has it; everyone has it after the heal	success = true, acks_received = 1; A, B: not found, C: value; converged after the heal	pass
P5	`test_partition_recovery_merges_divergent_values_with_lww`	Partition. `ALWAYS_AP` via A, then via C; heal.	acks 2 and 1; the replicas diverge with concurrent vector clocks; after the heal all keep the write with the higher timestamp	acks 2 and 1; A, B = dc1 value, C = dc2 value, vector clocks concurrent; after the heal all hold the dc2 value; LINEARIZABLE via B returns it	pass
P6	`test_lww_follows_the_timestamp_not_the_arrival_order`	Partition. Via C with timestamp now + 2 s, then via A with now + 1 s; heal.	the larger timestamp wins although it was written first	every replica holds the value written through C (first, but with the newer timestamp)	pass
P7	`test_union_merge_keeps_both_concurrent_writes`	Cluster with --merge union. "milk"; partition; "milk,eggs" via A, "milk,bread" via C; heal.	both concurrent writes survive	"bread,eggs,milk" on every replica	pass
P8	`test_lww_with_clock_skew_keeps_the_older_write`	C's clock 3 s behind. Partition; write via A, 200 ms later via C; heal.	LWW keeps the earlier write (wrong winner caused by the skew)	"earlier-write-dc1" everywhere	pass
R1	`test_linearizable_read_is_answered_by_the_leaseholder`	`QUORUM_CP` via B; `LINEARIZABLE` via A, B, C.	value v1, served by A, is_stale = false	v1, served by A, is_stale = false for all three gateways	pass
R2	`test_linearizable_read_fails_on_the_minority_side`	Partition {A, B} | {C}. `LINEARIZABLE` via C and via B.	via C: `LEASEHOLDER_UNAVAILABLE` although C has a copy; via B: the value	via C: error code `LEASEHOLDER_UNAVAILABLE` (raised as `ReplicaUnavailableError`); via B: v1	pass
R3	`test_follower_read_is_served_locally_with_bounded_lag`	Healthy cluster. `FOLLOWER_STALE` via B.	served by B, is_stale = true, lag < 1 s, 0 < read_timestamp <= now	as expected (the same read in step 2 of the demo run: lag 25 ms)	pass
R4	`test_follower_read_stays_available_but_stale_during_partition`	v1 everywhere; partition; `QUORUM_CP` v2 via A; `FOLLOWER_STALE` via C; `LINEARIZABLE` via B.	C answers v1 with is_stale = true; B answers v2	C: v1, is_stale = true, served by C; B: v2	pass
R5	`test_follower_read_beyond_max_staleness_is_rejected`	Partition; wait 0.8 s; `FOLLOWER_STALE` via C with 300 ms and with 5000 ms; heal.	300 ms: `STALE_READ_EXCEEDED`; 5000 ms: answered; after the heal 300 ms works again	300 ms: `FAILED_PRECONDITION` "`STALE_READ_EXCEEDED`: follower C lags ... ms"; 5000 ms: v1; after the heal: v1	pass
Q1-Q9	`test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n [W1-R1 ... W3-R3]`	The write reaches exactly W replicas; every read set of size R is asked.	stale read sets = C(3 - W, R)	as predicted in all nine cases (Table 5)	pass

Id	Test	Scenario	Expected result	Actual result	Result
C1	`test_follower_crash_then_restart_catches_up`	Kill C; `QUORUM_CP` via A; restart C (empty store).	commit with acks_received = 2; C catches up and serves follower reads	acks_received = 2; C holds the value after the restart	pass
C2	`test_leaseholder_crash_blocks_cp_but_not_follower_reads_or_ap`	Kill A. Via B: `QUORUM_CP`, `LINEARIZABLE`, `FOLLOWER_STALE`, `ALWAYS_AP`; restart A.	CP and linearizable fail; follower read v1; AP accepted with 2 acks; the restarted A recovers	as expected; A serves "v3-ap" after the full state of B and C arrived	pass
C3	`test_retry_with_same_idempotency_key_is_applied_once`	`QUORUM_CP` twice with one idempotency key, via B and then via C.	the second reply equals the first; no new version	same acks; version timestamp on A unchanged	pass
C4	`test_key_outside_the_leaseholder_range_is_rejected`	Put "Cart", Get "42".	`InvalidKeyError` (only [a-z] exists)	`InvalidKeyError` for both	pass

Table 4

Task 3: Partition Handling, Handout Expectation Against Observation

Step	Request	Expected (handout)	Observed	Evidence
Part A.1	QUORUM_CP to DC-1 (A, B)	Success, acks_received = 2	success = true, acks_received = 2	P1; Appendix D, step 4
Part A.2	QUORUM_CP to DC-2 (C)	Failure (ReplicaUnavailableError / quorum loss)	success = false, acks_received = 0, ReplicaUnavailableError: quorum lost - leaseholder A is unreachable from C	P2; Appendix D, step 4
Part B.1	ALWAYS_AP to DC-2 (C)	Write accepted, success = true, acks_received = 1	success = true, acks_received = 1	P4; Appendix D, step 6
Part B.2	Heal the partition, then read	LWW (by timestamp) merges the divergent values	all replicas converge to the version with the higher timestamp within about 0.5 s; a linearizable read returns it	P5, P6; Appendix D, step 8

Note. C reports 0 acknowledgements because it is not the leaseholder: it forwards the write to A, cannot reach it, and no replica stages the write. When the leaseholder itself is cut off (P3) the count is 1, its own copy.

Quorum verification

For every combination of W and R from 1 to 3 a fresh key is first written to all three replicas ("old"). Then the partition cuts the first W replicas (A, then B, then C) off from the rest, so an `ALWAYS_AP` write of "new" through A reaches exactly W of them; `acks_received` confirms it. While the partition holds, every possible read set of R replicas is asked for its local version, and the quorum read returns the one with the highest timestamp. A read set is stale if that is still "old". Theory: a read set misses the write only if all its R replicas lie outside the write set, which gives C(N - W, R) stale read sets, zero exactly when R + W > N.

Table 5

Quorum Verification: Stale Read Sets for Every W and R (N = 3)

W	R	R + W	R + W > N	Write acks	Read sets tried	Stale read sets (measured)	Predicted C(N-W, R)	Stale read sets
1	1	2	no	1	3	2	2	{B} {C}
1	2	3	no	1	3	1	1	{B,C}
1	3	4	yes	1	1	0	0	-
2	1	3	no	2	3	1	1	{C}
2	2	4	yes	2	3	0	0	-
2	3	5	yes	2	1	0	0	-
3	1	4	yes	3	3	0	0	-
3	2	5	yes	3	3	0	0	-
3	3	6	yes	3	1	0	0	-

Note. Measured by demo.py (Appendix D, step 9) and asserted by tests Q1-Q9. Stale reads occur only in the three combinations with R + W <= N.

2. Performance Evaluation (PACELC)

`perf_cap_bench.py` starts three replica processes on localhost. The client sits next to follower C: it sends every request to C, so leaseholder A is the remote replica. Each scenario is 1,000 sequential Get requests after 100 warm-up requests; the latency is measured on the client with `time.perf_counter`. Median and p95 (the value at position ceil(0.95 n) of the sorted list) are taken over the successful requests; availability is successful requests / 1,000. The script measures three phases: the healthy cluster; the partition DC-1 {A, B} | DC-2 {C} during its first 0.8 s; and the partition after C's lag passed 5000 ms.

Table 6

PACELC Table From the Handout, Filled In

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%) healthy / partition
Linearizable	Leaseholder (Remote)	0.73	0.89	100.0 / 0.0
Follower Stale	Local Follower	0.34	0.43	100.0 / 100.0

Note. Latency from the healthy cluster. Availability first in the healthy cluster, then during the partition with C cut off from A. FOLLOWER_STALE uses max_staleness_ms = 5000. Raw output in Appendix C.

Table 7
All Scenarios of the Localhost Run (1,000 Requests Each)

Scenario	Network	OK	Avail. (%)	Median (ms)	p95 (ms)	Errors
Linearizable via C to leaseholder A	healthy	1000	100.0	0.73	0.89	-
Follower stale on C	healthy	1000	100.0	0.34	0.43	-
Linearizable via A (client next to A)	healthy	1000	100.0	0.34	0.52	-
Linearizable via C to leaseholder A	partition, first 0.8 s	0	0.0	-	-	1000 x `LEASEHOLDER_UNAVAILABLE`
Follower stale on C	partition, first 0.8 s	1000	100.0	0.35	0.47	-
Follower stale on C	partition, C lag 5043 ms	0	0.0	-	-	1000 x `STALE_READ_EXCEEDED`

Table 8
The Same Benchmark With 5 ms Simulated Delay Between the Replicas

Scenario	Network	OK	Avail. (%)	Median (ms)	p95 (ms)	Errors
Linearizable via C to leaseholder A	healthy	1000	100.0	11.84	13.54	-
Follower stale on C	healthy	1000	100.0	0.35	0.49	-
Linearizable via A (client next to A)	healthy	1000	100.0	0.34	0.41	-
Linearizable via C to leaseholder A	partition, first 0.8 s	0	0.0	-	-	1000 x `LEASEHOLDER_UNAVAILABLE`
Follower stale on C	partition, first 0.8 s	1000	100.0	0.34	0.42	-

Note. python perf_cap_bench.py --link-delay-ms 5: every replica-to-replica proxy delays each chunk of data by 5 ms in each direction, as if A were in another data centre. Client-to-replica connections stay local.

Else (no partition): latency against consistency. A linearizable read sent to C needs two hops (client to C, C to A and back), a follower read one. On one machine the median is therefore 2.1 times higher (0.73 ms against 0.34 ms). The extra cost is the distance to the leaseholder, not the read: a linearizable read sent directly to A costs 0.34 ms. With 5 ms of delay per direction between the replicas the gap grows to 34 times (11.84 ms against 0.35 ms); the follower read does not change, because it never leaves C.

Partition: availability against consistency. With DC-2 cut off, all 1,000 linearizable reads on C failed immediately with `LEASEHOLDER_UNAVAILABLE` (0 %), while all 1,000 follower reads succeeded (100 %) at the usual latency, returning the state from the moment the partition began. The bound limits how old that state may be: once C's lag reached 5043 ms (above 5000 ms) every follower read was refused with `STALE_READ_EXCEEDED` (0 %). Bounded staleness thus returns the follower read from the AP side to the CP side once the data would be too old.

3. Conceptual Questions

Question 1. Why does R + W > N guarantee linearizability, whereas R + W <= N allows stale reads?

A write is acknowledged by a set of W replicas and a read asks a set of R replicas. Among N replicas two such sets share at least R + W - N replicas. If R + W > N they share at least one, so every read quorum contains a replica that holds the latest acknowledged write. The reader compares the versions it gets and returns the one with the highest timestamp, so it cannot return a value older than that write. If R + W <= N the two sets can be disjoint: a read can be answered only by replicas that missed the write, and it returns the old value without noticing. The number of such read sets is C(N - W, R). The quorum experiment measured exactly this number in all nine cases (Table 5): zero for every combination with R + W > N, one or two for the three others.

Intersection is the core of the guarantee, but linearizability needs two more conditions. Writes must also intersect each other (W > N/2); otherwise two writes can be acknowledged by disjoint sets and their order is undefined. And once a read has returned the new value, no later read may return the old one while the write is still spreading; Dynamo-style quorums handle that with read repair, writing the newest version back to the read quorum before replying, as in the ABD algorithm. The CP path of this system meets both conditions the CockroachDB way: every `QUORUM_CP` write needs W = 2 of 3 and goes through the leaseholder, which applies it only after the commit, and linearizable reads are answered by the leaseholder. The leaseholder belongs to every write quorum, so reading it is the case R = 1 in which the one replica read is guaranteed to be in the intersection. Follower reads and `ALWAYS_AP` writes give the guarantee up: W = 1 on one replica and R = 1 on another is R + W = 2 <= 3. That is why C answered "milk" while DC-1 had already committed "milk,eggs" (Appendix D, step 5).

Question 2. In the partition test, what trade-off did the system make when handling writes on the minority partition under QUORUM_CP and under ALWAYS_AP?

Under `QUORUM_CP` the system chose consistency over availability. C could reach neither the leaseholder nor a majority, so it rejected the write (success = false, ReplicaUnavailableError) and stored nothing. A client in DC-2 could not write, and could not read linearizably either, until the partition healed. In return the data never diverged: after the heal there was nothing to reconcile, and DC-1 kept accepting writes with two acknowledgements. Only the majority side stays available, and only because it holds the leaseholder: when A itself was cut off (test P3) CP writes stopped everywhere, because the lease in this emulation never moves.

Under `ALWAYS_AP` the system chose availability. C accepted the write with acks_received = 1 and the client got an immediate success. The price was divergence: while the partition lasted, A and B held one value and C another, with concurrent vector clocks, and the answer to a read depended on the replica that served it. After the heal, anti-entropy had to

resolve the conflict. Last-write-wins made all replicas agree on the version with the higher timestamp, but it silently discarded the other acknowledged write ("milk,eggs" in Appendix D, step 8), and it trusts the clocks: with C's clock 3 s behind, the write that happened later lost (test P8). Union merge kept both writes of the shopping-cart value ("bread,eggs,milk", test P7), but it only works for data that can be merged as a set, and it cannot express a deletion.

In PACELC terms the `QUORUM_CP` + `LINEARIZABLE` path is PC/EC: unavailable in the minority partition, and it pays the extra hop to the leaseholder even without a partition. The `ALWAYS_AP` + `FOLLOWER_STALE` path is PA/EL: available and fast, but stale and in need of conflict resolution, with `max_staleness_ms` limiting how stale a follower read may be.

4. Limitations

- The lease never moves. CockroachDB would let the lease expire and move it to the majority side; here a lost leaseholder stops CP writes and linearizable reads in the whole cluster.
- The replicas keep their data in memory. A restarted replica rebuilds its store from the others through anti-entropy; a restarted leaseholder serves again only after every follower has sent it the full state.
- All replicas share one machine and one clock. Clock skew is simulated (--clock-skew-ms), and WAN latency with the proxy delay (--link-delay-ms); the localhost latencies are lower bounds.

Appendix A. How to Reproduce

```
cd lab6_7
..\.venv\Scripts\activate
pip install -r requirements.txt
pytest -v                                # 26 tests, Appendix B
python perf_cap_bench.py                 # Tables 6 and 7, Appendix C
python perf_cap_bench.py --link-delay-ms 5 # Table 8
python demo.py                           # walkthrough, Appendix D
python demo.py --interactive             # own experiments: put, get, partition, heal, crash ...
```

Appendix B. pytest Output

```
============================= test session starts =============================
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\askha\PycharmProjects\DS_midterm\.
      venv\Scripts\python.exe
rootdir: C:\Users\askha\PycharmProjects\DS_midterm\lab6_7
configfile: pytest.ini
testpaths: tests
collecting ... collected 26 items

tests/test_cap.py::test_cp_write_on_majority_side_succeeds PASSED        [  3%]
tests/test_cap.py::test_cp_write_on_minority_side_fails PASSED           [  7%]
tests/test_cap.py::test_isolated_leaseholder_cannot_commit_alone PASSED  [ 11%]
tests/test_cap.py::test_ap_write_on_minority_side_is_accepted PASSED     [ 15%]
tests/test_cap.py::test_partition_recovery_merges_divergent_values_with_lww PASSED [ 19%]
tests/test_cap.py::test_lww_follows_the_timestamp_not_the_arrival_order PASSED [ 23%]
tests/test_cap.py::test_union_merge_keeps_both_concurrent_writes PASSED  [ 26%]
tests/test_cap.py::test_lww_with_clock_skew_keeps_the_older_write PASSED [ 30%]
tests/test_cap.py::test_linearizable_read_is_answered_by_the_leaseholder PASSED [ 34%]
tests/test_cap.py::test_linearizable_read_fails_on_the_minority_side PASSED [ 38%]
tests/test_cap.py::test_follower_read_is_served_locally_with_bounded_lag PASSED [ 42%]
tests/test_cap.py::test_follower_read_stays_available_but_stale_during_partition PASSED [ 46%]
tests/test_cap.py::test_follower_read_beyond_max_staleness_is_rejected PASSED [ 50%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W1-R1] PASSED [ 53%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W1-R2] PASSED [ 57%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W1-R3] PASSED [ 61%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W2-R1] PASSED [ 65%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W2-R2] PASSED [ 69%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W2-R3] PASSED [ 73%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W3-R1] PASSED [ 76%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W3-R2] PASSED [ 80%]
tests/test_cap.py::test_read_quorum_sees_latest_write_only_if_r_plus_w_exceeds_n[W3-R3] PASSED [ 84%]
tests/test_cap.py::test_follower_crash_then_restart_catches_up PASSED    [ 88%]
tests/test_cap.py::test_leaseholder_crash_blocks_cp_but_not_follower_reads_or_ap PASSED [ 92%]
tests/test_cap.py::test_retry_with_same_idempotency_key_is_applied_once PASSED [ 96%]
tests/test_cap.py::test_key_outside_the_leaseholder_range_is_rejected PASSED [100%]

============================== 26 passed in 8.92s ==============================
```

Appendix C. Benchmark Output

```
PACELC read benchmark (Task 4)
3 replicas on localhost, 1000 sequential Get requests per scenario, client next to follower C.
Leaseholder A is the remote node. FOLLOWER_STALE uses max_staleness_ms = 5000.
Machine: Windows-11-10.0.26200-SP0, Python 3.14.6

Table 1. Lab table. Latency: healthy cluster. Availability: healthy / during the partition.
| Read Mode | Target Node | Median Latency (ms) | p95 Latency (ms) | Read Availability (%) |
|---|---|---|---|---|
| Linearizable | Leaseholder (Remote) | 0.73 | 0.89 | 100.0 / 0.0 |
| Follower Stale | Local Follower | 0.34 | 0.43 | 100.0 / 100.0 |

Table 2. Every scenario in detail.
| Scenario | Network | Requests | Successful | Availability (%) | Median (ms) | p95 (ms) | Errors |
|---|---|---|---|---|---|---|---|
| Linearizable via C -> leaseholder A | healthy | 1000 | 1000 | 100.0 | 0.73 | 0.89 | - |
| Follower stale on C | healthy | 1000 | 1000 | 100.0 | 0.34 | 0.43 | - |
| Linearizable via A (client next to leaseholder) | healthy | 1000 | 1000 | 100.0 | 0.34 | 0.52 | - |
| Linearizable via C -> leaseholder A | partition, first 0.8 s | 1000 | 0 | 0.0 | - | - |
      LEASEHOLDER_UNAVAILABLE x1000 |
| Follower stale on C | partition, first 0.8 s | 1000 | 1000 | 100.0 | 0.35 | 0.47 | - |
| Follower stale on C | partition, C lag 5043 ms > 5000 ms | 1000 | 0 | 0.0 | - | - | STALE_READ_EXCEEDED
      x1000 |

Linearizable via C costs 2.1x the median latency of a follower read (an extra hop to the leaseholder).

PACELC read benchmark (Task 4)
3 replicas on localhost, 1000 sequential Get requests per scenario, client next to follower C.
Leaseholder A is the remote node. FOLLOWER_STALE uses max_staleness_ms = 5000.
Simulated WAN: every replica-to-replica link adds 5 ms per direction.
```

Machine: Windows-11-10.0.26200-SP0, Python 3.14.6

Table 1. Lab table. Latency: healthy cluster. Availability: healthy / during the partition.

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder (Remote)	11.84	13.54	100.0 / 0.0
Follower Stale	Local Follower	0.35	0.49	100.0 / 100.0

Table 2. Every scenario in detail.

Scenario	Network	Requests	Successful	Availability (%)	Median (ms)	p95 (ms)	Errors
Linearizable via C -> leaseholder A	healthy	1000	1000	100.0	11.84	13.54	-
Follower stale on C	healthy	1000	1000	100.0	0.35	0.49	-
Linearizable via A (client next to leaseholder)	healthy	1000	1000	100.0	0.34	0.41	-
Linearizable via C -> leaseholder A	partition, first 0.8 s	1000	0	0.0	-	-	LEASEHOLDER_UNAVAILABLE x1000
Follower stale on C	partition, first 0.8 s	1000	1000	100.0	0.34	0.42	-
Follower stale on C	partition, C lag 5039 ms > 5000 ms	1000	0	0.0	-	-	STALE_READ_EXCEEDED x1000

Linearizable via C costs 34.0x the median latency of a follower read (an extra hop to the leaseholder).

Appendix D. Demo Output (python demo.py), Steps 3 to 9

```
================================================================================================
Step 3. Network partition: DC-1 {A, B} | DC-2 {C}
================================================================================================
  partitioned: {A, B} | {C}
    | 22:29:17.707394 [A] NET      peer C is UNREACHABLE
    | 22:29:17.769221 [C] NET      peer A is UNREACHABLE
    | 22:29:17.769338 [B] NET      peer C is UNREACHABLE
    | 22:29:17.769419 [C] NET      peer B is UNREACHABLE

================================================================================================
Step 4. CP mode under the partition (Task 3, Part A)
================================================================================================
  Majority side: 2 of 3 replicas acknowledge -> committed.

  > put cart='milk,eggs' QUORUM_CP via A
    OK    success=True, acks_received=2/3
    | 22:29:17.857732 [B] PREPARE  cart='milk,eggs' staged for leaseholder A -> ack
    | 22:29:17.858081 [A] APPLY    cart='milk,eggs' ts=22:29:17.857 vc={A@22:29:17.857} [commit]
    | 22:29:17.858130 [A] PUT      cart='milk,eggs' QUORUM_CP ts=22:29:17.857 -> COMMIT, acks 2/3
    | 22:29:17.858604 [B] APPLY    cart='milk,eggs' ts=22:29:17.857 vc={A@22:29:17.857} [sync from A]
  Minority side: C cannot reach the leaseholder or a majority -> rejected, nothing written.

  > put cart='milk,bread' QUORUM_CP via C
    FAIL  success=False, acks_received=0: ReplicaUnavailableError: quorum lost - leaseholder A is
      unreachable from C, so no majority (2 of 3) can acknowledge the write
    | 22:29:18.024629 [C] PUT      cart='milk,bread' QUORUM_CP -> FAILED, ReplicaUnavailableError: quorum
      lost - leaseholder A is unreachable from C, so no majority (2 of 3) can acknowledge the write

================================================================================================
Step 5. Reads during the partition (PACELC: if Partition, choose A or C)
================================================================================================

  > get cart LINEARIZABLE via C
    FAIL  ReplicaUnavailableError: LEASEHOLDER_UNAVAILABLE: leaseholder A is unreachable from C, a
      linearizable read is not possible on this side of the partition
    | 22:29:18.184716 [C] GET      cart LINEARIZABLE -> leaseholder A unreachable -> UNAVAILABLE
  C refuses the linearizable read (consistency), but a follower read stays available:

  > get cart FOLLOWER_STALE max_staleness_ms=5000 via C
    OK    value='milk'  served by C, is_stale=True, lag 746 ms, read_ts=22:29:17.599, 0.87 ms
    | 22:29:18.345812 [C] GET      cart FOLLOWER_STALE on follower (lag 746 ms <= 5000 ms) -> 'milk'
  The value is old: 'milk,eggs' was committed in DC-1 and C cannot know it.

================================================================================================
Step 6. AP mode under the partition (Task 3, Part B)
================================================================================================
  ALWAYS_AP is accepted by the one replica that can be reached (sloppy quorum).

  > put cart='milk,bread' ALWAYS_AP via C
    OK    success=True, acks_received=1/3
    | 22:29:18.506741 [C] APPLY    cart='milk,bread' ts=22:29:18.506 vc={A@22:29:17.162, C@22:29:18.506} [
      local write]
    | 22:29:18.507167 [C] PUT      cart='milk,bread' ALWAYS_AP ts=22:29:18.506 vc={A@22:29:17.162, C@22:
      29:18.506} -> ACCEPTED, acks 1/3 (sloppy quorum: the other replicas get it after the partition
      heals)

  state of 'cart' on every replica:
    A  'milk,eggs'      ts=22:29:17.857  vc={A@22:29:17.857}
    B  'milk,eggs'      ts=22:29:17.857  vc={A@22:29:17.857}
    C  'milk,bread'     ts=22:29:18.506  vc={A@22:29:17.162, C@22:29:18.506}
  The replicas now disagree: two concurrent versions (look at the vector clocks).

================================================================================================
Step 7. Bounded staleness: C stops hearing from the leaseholder
================================================================================================

  > get cart FOLLOWER_STALE max_staleness_ms=1500 via C
    FAIL  StaleReadExceededError: STALE_READ_EXCEEDED: follower C lags 1531 ms behind the leaseholder,
      max_staleness_ms=1500
    | 22:29:19.130303 [C] GET      cart FOLLOWER_STALE: follower lags 1531 ms behind the leaseholder,
      max_staleness_ms=1500 -> STALE_READ_EXCEEDED
  With a looser bound the same follower still answers:

  > get cart FOLLOWER_STALE max_staleness_ms=5000 via C
    OK    value='milk,bread'  served by C, is_stale=True, lag 1694 ms, read_ts=22:29:17.599, 1.11 ms
    | 22:29:19.293654 [C] GET      cart FOLLOWER_STALE on follower (lag 1694 ms <= 5000 ms) -> 'milk,
      bread'

================================================================================================
Step 8. The partition heals: anti-entropy + last-write-wins
================================================================================================
```

```
    | 22:29:19.594155 [C] MERGE    cart: conflict, last-write-wins -> kept 'milk,bread' (ts 22:29:18.506),
       dropped 'milk,eggs' from A (ts 22:29:17.857) [sync from A]
    | 22:29:19.594430 [A] NET      peer C is reachable again
    | 22:29:19.717616 [B] MERGE    cart: conflict, last-write-wins -> 'milk,bread' (ts 22:29:18.506),
      local was 'milk,eggs' (ts 22:29:17.857) [sync from C]
    | 22:29:19.718025 [C] NET      peer B is reachable again
    | 22:29:19.825056 [A] MERGE    cart: conflict, last-write-wins -> 'milk,bread' (ts 22:29:18.506),
      local was 'milk,eggs' (ts 22:29:17.857) [sync from B]
    | 22:29:19.825302 [C] NET      peer A is reachable again
    | 22:29:20.039189 [B] NET      peer C is reachable again

  state of 'cart' on every replica:
    A  'milk,bread'     ts=22:29:18.506  vc={A@22:29:17.162, C@22:29:18.506}
    B  'milk,bread'     ts=22:29:18.506  vc={A@22:29:17.162, C@22:29:18.506}
    C  'milk,bread'     ts=22:29:18.506  vc={A@22:29:17.162, C@22:29:18.506}
  All replicas converged to 'milk,bread': the concurrent write with the higher timestamp.
  The other one ('milk,eggs' from DC-1) is silently lost - the price of LWW.

  > get cart LINEARIZABLE via C
    OK    value='milk,bread'  served by A, is_stale=False, read_ts=22:29:20.061, 1.01 ms
    | 22:29:20.061082 [A] GET      cart LINEARIZABLE on leaseholder -> 'milk,bread'
    | 22:29:20.061339 [C] GET      cart LINEARIZABLE -> proxied to leaseholder A -> 'milk,bread'

================================================================================================
Step 9. Quorum arithmetic: does a read quorum of R see a write acknowledged by W?
================================================================================================

  N = 3. The write reaches exactly W replicas (partition), then every read set of size R
  is asked; it returns the version with the highest timestamp.

    W  R  R+W  R+W>N   write acks  read sets  stale read sets  which
    1  1  2    no      1           3          2                {B} {C}
    1  2  3    no      1           3          1                {B,C}
    1  3  4    yes     1           1          0                -
    2  1  3    no      2           3          1                {C}
    2  2  4    yes     2           3          0                -
    2  3  5    yes     2           1          0                -
    3  1  4    yes     3           3          0                -
    3  2  5    yes     3           3          0                -
    3  3  6    yes     3           1          0                -

  Stale reads happen only when R + W <= N: then a read set can miss every replica of the write.
```